NESNE TABANLI PROGRAMLAMA

Laboratuvar Föyü

# Hafta 5 — Sınıflar, Yapıcı Metotlar ve Statik Üyeler

*Öğrenci Yönergesi — Adım Adım Uygulama*

| Bilgi | Değer |
|---|---|
| Ders | Nesne Tabanlı Programlama (2. Sınıf) |
| Konu | Sınıf, property (get/set), yapıcı metot, statik üye |
| Ortam | Visual Studio Community 2026 · .NET 10 (LTS) |
| Dil | C# |
| Süre | Yaklaşık 2–3 saat |
| Teslim | Word,pdf |

## 1. Bu Haftanın Hedefi

Bu hafta nesne tabanlı programlamanın kalbine giriyoruz: kendi sınıfımızı tasarlayıp, o sınıfın verisini property (get/set) ile koruyacak, nesne oluşturulurken otomatik çalışan yapıcı metotlar (constructor) yazacak ve tüm nesneler için ortak olan statik üyeleri kullanacağız. Uygulama olarak bir "Ev" ve içindeki kişileri modelleyeceğiz.

## 2. Kazanımlar

Bu haftanın sonunda şunları yapabilir hâle geleceksiniz:

- Erişim belirleyicileri (public, private) ile sınıf tasarlamak,
- property (get/set) tanımlayıp set içinde doğrulama yapmak,
- yapıcı metot (constructor) yazarak nesneyi başlangıç değerleriyle oluşturmak,
- nesneye ait metot yazmak (ör. yaş hesaplama),
- statik üye (static) tanımlayıp SınıfAdı.üye biçiminde erişmek.

## 3. Temel Kavramlar

### 3.1. Sınıf ve Erişim Belirleyiciler

Sınıf (class), gerçek hayattaki bir nesneyi modelleyen yapıdır; içinde özellikler (veri) ve metotlar (davranış) bulunur. Erişim belirleyiciler bir üyeye nereden erişilebileceğini tanımlar:

- **public:** sınıfın dışından da erişilebilir.
- **private:** yalnızca sınıfın içinden erişilebilir. (Erişim belirleyici yazılmazsa varsayılan private'tır.)

### 3.2. Property (get/set)

Bir private alana kontrollü erişim sağlar. get değeri okur, set yeni değeri atar; set içinde value gelen değeri temsil eder ve doğrulama burada yapılır:

```
private int dogumYili;
public int DogumYili
{
    get { return dogumYili; }
```

```
    set
    {
        if (value > 1900 && value <= DateTime.Now.Year)
            dogumYili = value;
        else
            Console.WriteLine("Geçersiz doğum yılı!");
    }
}
```

### 3.3. Yapıcı Metot (Constructor)

Nesne new ile oluşturulurken otomatik çalışan özel metottur. Sınıfla aynı adı taşır, dönüş tipi yoktur (void bile yazılmaz) ve başlangıç değerlerini atar.

```
public Kisi(string ad, int dogumYili, string meslek)
{
    Ad = ad;
    DogumYili = dogumYili;   // property set bloğundan geçer (doğrulama)
    Meslek = meslek;
}
```

### 3.4. Statik Üye (static)

Bir sınıfın tüm nesneleri için ortak olan, nesneye değil sınıfa ait üyedir. Nesne oluşturmadan SınıfAdı.üye biçiminde erişilir. Örneğin kaç kişi oluşturulduğunu sayan bir sayaç statik olmalıdır.

**Statik üyeye nasıl erişilir?**
Statik üyeye nesne üzerinden değil, sınıf adıyla erişilir: Kisi.KisiSayisi
Normal (statik olmayan) üyeye ise nesne üzerinden erişilir: kisi.Ad

## 4. Uygulama: Evim

Bir evi ve içindeki kişileri modelleyeceğiz. Kisi sınıfı bir kişinin adını, mesleğini ve doğum yılını tutacak, yaşını hesaplayacak; Ev sınıfı ise bu kişileri bir arada tutacak.

Önce **Hafta5_Evim** adında bir Konsol Uygulaması (C#, .NET 10) oluşturun. Sonra iki sınıf dosyası ekleyin: **Kisi.cs** ve **Ev.cs**.

### Adım A — Kisi sınıfı: property ve statik sayaç

Doğum yılını private bir alanda tutup property ile doğrulayarak sunuyoruz. Ad ve Meslek için auto-property yeterli. KisiSayisi ise statik bir sayaç:

```
using System;

namespace Hafta5_Evim
{
    class Kisi
    {
        public static int KisiSayisi = 0;   // statik: tüm nesneler için ortak

        private int dogumYili;

        public string Ad { get; set; }
        public string Meslek { get; set; }

        public int DogumYili
        {
            get { return dogumYili; }
            set
            {
                if (value > 1900 && value <= DateTime.Now.Year)
```

```
                    dogumYili = value;
                else
                    Console.WriteLine("Geçersiz doğum yılı!");
            }
        }
    }
}
```

### Adım B — Yapıcı metot ve nesneye ait metotlar

Kisi sınıfının içine; başlangıç değerlerini atayan yapıcı metodu, yaşı hesaplayan metodu ve bilgiyi yazan metodu ekleyin:

```
public Kisi(string ad, int dogumYili, string meslek)
{
    Ad = ad;
    DogumYili = dogumYili;
    Meslek = meslek;
    KisiSayisi++;          // her yeni kişi oluştukça artar
}

public int YasHesapla()
{
    return DateTime.Now.Year - dogumYili;
}

public void BilgiYaz()
{
    Console.WriteLine($"{Ad} - {Meslek} - {YasHesapla()} yaşında");
}
```

### Adım C — Ev sınıfı: kişileri bir arada tutma

Ev sınıfı bir Kisi listesi tutar; kişi ekler, hepsini listeler ve ortalama yaşı hesaplar:

```
using System;
using System.Collections.Generic;

namespace Hafta5_Evim
{
    class Ev
    {
        private List<Kisi> uyeler = new List<Kisi>();

        public void UyeEkle(Kisi k)
        {
            uyeler.Add(k);
        }

        public void TumUyeleriYaz()
        {
            foreach (Kisi k in uyeler)
                k.BilgiYaz();
        }

        public double OrtalamaYas()
        {
            if (uyeler.Count == 0) return 0;
            int toplam = 0;
            foreach (Kisi k in uyeler)
                toplam += k.YasHesapla();
            return (double)toplam / uyeler.Count;
        }
    }
}
```

### Adım D — Main'de evi kurma

```
using System;

namespace Hafta5_Evim
{
    class Program
    {
        static void Main(string[] args)
        {
            Ev evim = new Ev();

            evim.UyeEkle(new Kisi("Anne", 1970, "Öğretmen"));
            evim.UyeEkle(new Kisi("Baba", 1968, "Mühendis"));
            evim.UyeEkle(new Kisi("Ben", 2004, "Öğrenci"));

            Console.WriteLine("--- EV ÜYELERİ ---");
            evim.TumUyeleriYaz();

            Console.WriteLine($"Ortalama yaş : {evim.OrtalamaYas():F1}");
            Console.WriteLine($"Toplam kişi  : {Kisi.KisiSayisi}");
        }
    }
}
```

**Ne görmelisiniz? (yaşlar içinde bulunulan yıla göre hesaplanır)**

--- EV ÜYELERİ ---
Anne - Öğretmen - 56 yaşında
Baba - Mühendis - 58 yaşında
Ben - Öğrenci - 22 yaşında
Ortalama yaş : 45,3
Toplam kişi  : 3

**⚠Statik üyeye dikkat**

Kisi.KisiSayisi yazdık, kisi.KisiSayisi değil. Çünkü KisiSayisi statiktir; nesneye değil sınıfa aittir.
Her new Kisi(...) çağrısı KisiSayisi'nı 1 artırdığı için sonuç 3 olur.

## 5. Kendi Göreviniz (Lab Teslimi)

Uygulamayı aşağıdaki gibi genişletin. Teslim edilecek çalışma budur:

1. Meslek'i auto-property yerine doğrulamalı property yapın: boş bir meslek girilirse "Belirtilmedi" atansın (set içinde kontrol).
2. Ev sınıfına en yaşlı üyeyi bulup döndüren bir metot ekleyin: public Kisi EnYasli().
3. Kisi sınıfına ev adresini tutan bir const veya static readonly alan ekleyin (ör. EvAdresi) ve Main'de yazdırın.
4. En az 4 kişi ekleyip; tüm üyeleri, ortalama yaşı, en yaşlı kişiyi ve toplam kişi sayısını ekrana yazdırın.

## 6. Sık Yapılan Hatalar

- **Statik üyeye nesneyle erişmek:** kisi.KisiSayisi hata verir. Statik üyeye Kisi.KisiSayisi biçiminde, sınıf adıyla erişilir.
- **Constructor'a dönüş tipi yazmak:** Yapıcı metoda void veya int gibi bir dönüş tipi yazılmaz; adı sınıf adıyla birebir aynı olmalıdır.
- **Property içinde kendini çağırmak:** get { return DogumYili; } sonsuz döngüye yol açar. Property arka plandaki private alanı döndürmelidir (return dogumYili;).

- **Liste başlatmayı unutmak:** private List<Kisi> uyeler; deyip = new List<Kisi>(); yazmazsanız UyeEkle çağrısında NullReferenceException alırsınız.
- **using System; eksikliği:** DateTime ve Console için gereklidir; sınıf dosyalarının başında olmalıdır.

## 7. Değerlendirme

| Ölçüt | Puan |
|---|---|
| Proje derleniyor ve çalışıyor | 15 |
| Kisi sınıfı: property (get/set) + doğrulama | 20 |
| Yapıcı metot doğru tanımlı | 20 |
| YasHesapla ve BilgiYaz metotları çalışıyor | 15 |
| Ev sınıfı kişileri tutuyor (liste) ve listeliyor | 15 |
| Görev: EnYasli + statik/const kullanımı | 15 |
| TOPLAM | 100 |

## 8. Ödev (Gelecek Haftaya Hazırlık)

Kisi sınıfındaki yapıcı metodu birden çok biçimde yazmayı deneyin: biri sadece ad alan (Kisi(string ad)), diğeri ad + doğum yılı + meslek alan. Aynı isimli ama farklı parametreli bu yapıcılar, yapıcı metot aşırı yüklemesidir.

Dikkat ederseniz nesneler, farklı veri türlerini ve davranışları tek bir yapıda topluyor. **Gelecek hafta operatör aşırı yükleme, indeksleyiciler, yapılar (struct) ve enum sabitleri** gibi sınıfları daha güçlü kullanmanın yollarını göreceğiz.